DO NOW — RECALL

Recursion and algorithm tracing

Name: _____ Class: _____ Date: _____

1. What is an activation record?

2. When does code after recursive call execute?

3. What is auxiliary stack space for depth n?

4. Search for 12 in [2,4,6,8,10,12,14]. Midpoint indexes?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Saved data for an active function call.

2. Model answer

On return from the deeper call.

3. Model answer

O(n) call stack.

4. Model answer

3→5; index5 found.

By the end, you can…

• Trace recursive calls downward before calculating values on return.

• Name the base case and variable that progresses toward it.

• Recursion depth consumes call-stack memory even when total time is linear.

Vocabulary — know these terms

recursion — Subroutine calls itself.

base case — Stops additional calls.

call stack — Active subroutine frames.

unwinding — Returning from nested calls.

NEW LEARNING

Read, annotate and explain

Understand

Trace recursive calls downward before calculating values on return.

Apply

Name the base case and variable that progresses toward it.

Evaluate

Recursion depth consumes call-stack memory even when total time is linear.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



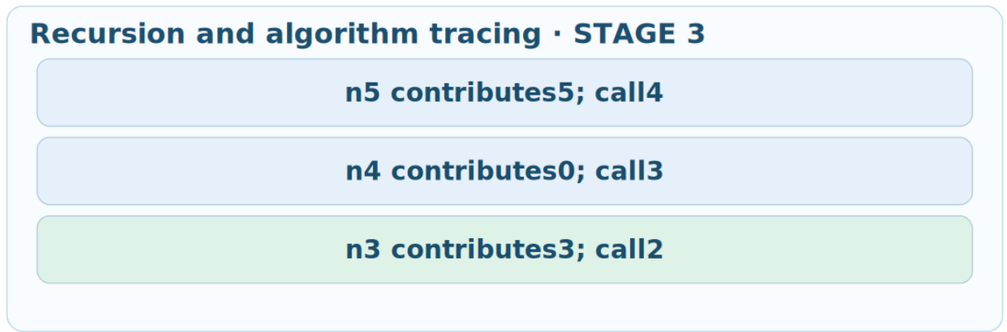
Recursion and algorithm tracing · STAGE 1

New Learning



Recursion and algorithm tracing · STAGE 2

New Learning



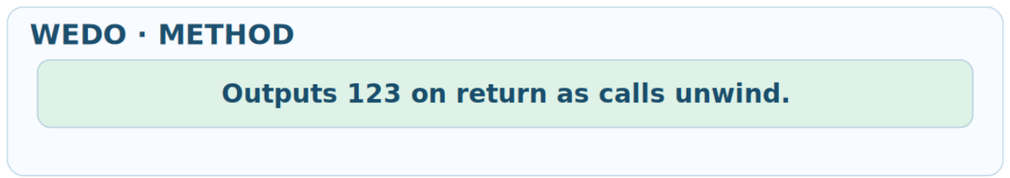
Recursion and algorithm tracing · STAGE 3

We Do source CHECK · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Predict output when printing after recursive call for n=3.

Outputs 123 on return as calls unwind.



WEDO · METHOD

WE DO — CHECK AND IMPROVE

Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

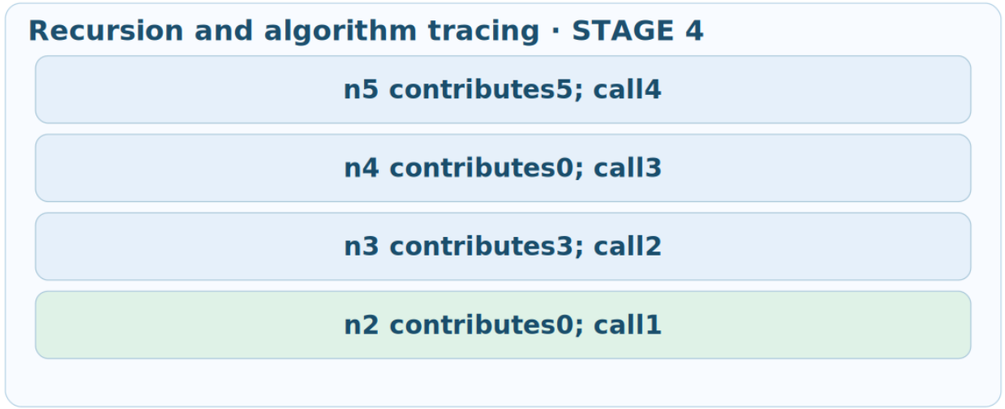
Guided 1 — CHECK

Explain the result
Outputs 123 on return as calls unwind.

New Learning · original lesson diagrams

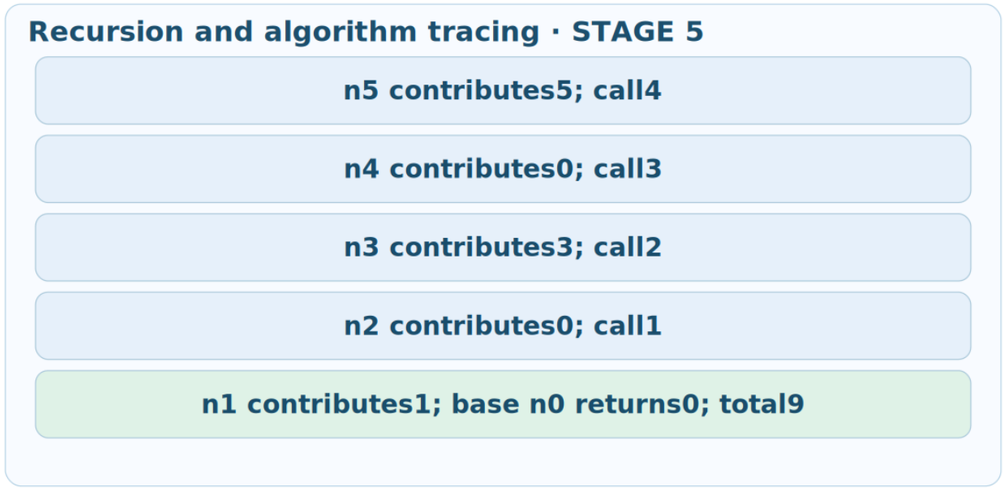
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Recursion and algorithm tracing · STAGE 4

New Learning



Recursion and algorithm tracing · STAGE 5

I DO — WORKED EXAMPLE

Follow the worked method

Predict output of printing before a recursive call for n=3.

Explain the result

Outputs 321 on descent. Nothing prints for n0.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

Printing after the recursive call appears on stack unwinding.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 3

Write an iterative C# method to sum the positive odd integers up to n (n ≥ 0), matching SumOdds(n). Compare its time and auxiliary space with the recursive method. [6]

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Predict output of printing before a recursive call for n=3.

Outputs 321 on descent. Nothing prints for n0.



IDO · METHOD

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

Guided 1 — TRY

Predict output when printing after recursive call for n=3.
Source code / input:

```
static void Count(int n){
if(n==0)return;
Count(n-1);
Console.Write(n);
}
```


YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

Trace SumOdds(4), listing contributions and returned result. [6]

Question 2

For F(n): if n=0 return, else print(n) then F(n-1), state F(4) output and why. [6]